운영체제 최종 설계 보고서

Child Process와 Multithread를 활용한 동기화 문제 해결 및 성능 분석

1조

팀원 : 남윤겸 박진서 안민석 이건아

담당 교수 : 이찬호 교수님

목차

1. 서론 4
    1.1 프로젝트 개요 4
    1.2 주제 선정 배경 및 동기 4
    1.3 설계 목표 5
    1.4 보고서 구성 6
2. 시스템 개요 7
    2.1 시스템 구성 및 동작 설명 7
        2.1.1 선택한 시스템의 역할과 기능 7
        2.1.2. 시스템 구성요소 분류 7
        2.1.3 전체 동작 순서도 8
    2.2 문제 분석 9
        2.2.1 병렬처리 가능 구성요소 파악 9
        2.2.2 External 연산 도입 배경 – 예비 실험을 통한 설계 문적 인식 9
        2.2.3 병목 구간 및 병렬화 제약 분석 10
    2.3 문제 해결 방향 11
        2.3.1 Process-Thread 대응 설계 11
        2.3.2 IPC(프로세스 간 통신)설계 11
        2.3.3 동기화(Synchronization) 필요 구간 식별 12
    2.4 최종 프로그램 구성 및 순서도 12
3. 프로그램 구성 14
    3.1 개발 환경 및 프로그램 구성 14
4. 동작 검증 및 성능 분석 15
    4.1 실험 설계 15
        4.1.1 성능 측정 지표 및 측정 방법 15
        4.1.2 Test Vector 설계 15
    4.2 실험 가. 순차 실행 vs Child Process 병렬처리 15
        4.2.1 실험 조건 15
        4.2.2 실험 결과 16
        4.2.3 분석 및 결론 17
    4.3 실험 나. 단일 Child Process vs 단일 Thread 17
        4.3.1 각 방식에 최적화된 코드 구성 17
        4.3.2 실험 결과 18
        4.3.3 분석 및 결론 20

4.4 실험 다. 다중 Child Process / 다중 Thread 확장 결과 21
4.4.1 실험 환경 및 조건 21
4.4.2 실제 성능 비교 및 정량 분석 21
4.4.3 성능 최적화 과정 및 임계점 검증 23
4.5 실험 라. 다중 Child Process + 다중 Thread 혼합 시스템 24
4.5.1 실험 목적 24
4.5.2 시스템 구성 및 동작 방식 24
4.5.3 실험 방법 25
4.5.4 실험 결과 및 분석 26
4.5.5 결과 분석 27
4.5.6 최적 구성 도출 28
4.5.7 시행착오 및 트러블슈팅 29
4.5.8 라 실험 결론 31
4.6 실험 마. Synchronization 적용 유무 비교 32
4.6.1 동기화 문제 발생 상황 유도 33
4.6.2 문제 해결 과정 33
4.6.3 동기화 적용 전후 성능 비교 34
5. 결론 36
5.1 프로젝트 성과 요약 36
5.2 한계 및 개선 방향 37
5.3 향후 확장 계획 38
6. 팀 운영 38
6.1 역할 분담 38
6.2 팀원별 기여도 자체 평가 40
6.3 수행 계획서 40
6.4 프로젝트 수행 일지 41
6. 사용한 AI agent 42

1. 서론

1.1 프로젝트 개요

본 프로젝트는 운영체제가 제공하는 프로세스(Process)와 스레드(Thread) 기반 병렬처리 기법을 실제 프로그램에 적용하고, 단계적 실험을 통해 CPU 코어 수에 비례하는 성능 향상을 달성하는 최적 시스템을 설계·도출하는 것을 목적으로 한다. 구현 대상으로는 HMM(Hidden Markov Model) 기반 대량 로그 분석 시스템을 선정하였다. 해당 시스템은 다수의 로그 파일을 입력받아 Viterbi 알고리즘으로 각 로그의 상태열을 추론하며, 전체 분석 소요 시간을 성능 지표로 삼는다.

로그 파일 단위의 분석은 상호 의존성이 없어 임의의 단위로 분할/병렬 처리가 가능하다. 본 프로젝트는 이러한 특성을 활용하여, 순차 실행을 기준(baseline)으로 삼고 child process와 thread를 단계적으로 도입/확장하면서 각 구조의 성능을 실험적으로 비교한다. 단일 실행 단위에서 출발하여 다중 실행 단위로 확장하고, 최종적으로 child process와 thread를 혼합한 구조에서 동기화(synchronization) 문제까지 해결하는 과정을 통해, 시스템 전체 차원에서 가장 효율적을 구성을 도출한다.

본 프로젝트가 다루는 핵심 운영체제 개념은 프로세스, 스레드, IPC(Inter-Process Communication), 동기화, 임계 구역(critical section), 스케줄링이며, 이들이 실제 프로그램의 성능에 미치는 영향을 실험 데이터로 검증하는 것이 본 프로젝트의 출발점이자 목표이다.

1.2 주제 선정 배경 및 동기

현대 컴퓨팅 환경에서는 서버, 보안 관제 시스템, 운영 로그 분석 도구 등 다양한 시스템에서 대량의 로그가 실시간으로 생성된다. 로그 수가 증가할수록 단일 프로세스 기반의 순차처리는 처리 시간이 선형적으로 증가하며, 다중 CPU 코어를 활용하지 못한다는 근본적 한계를 가진다.

본 프로젝트에서 HMM 기반 Viterbi 연산을 선택한 이유는 작업의 CPU-bound 특성에 있다. Viterbi 알고리즘은 반복적인 확률 계산을 수행하는 구조로, 로그 수와 연산 반복 횟수가 증가할수록 CPU 연산량이 비례하여 증가한다. 파일 입출력이 주를 이루는 I/O-bound 시스템은 디스크 병목으로 인해 병렬화 효과가 제한되지만, 본 시스템은 계산 비중이 크기 때문에 병렬처리 구조 간 성능 차이를 명확하게 측정하고, 코어 수에 비례하는 이상적 성능 향상을 검증하기에 적합하다.

주제 선정의 구체적 근거는 다음 네 가지로 정리된다.

- **실용성**: 로그 분석은 운영체제, 서버, 보안 시스템 전반에서 실제로 요구되는 작업이다. 가상의 연산이 아닌 현실적인 문제 구조를 채택함으로써 실험 결과가 실제 시스템 설계에 주는 시사점을 높인다.
- **병렬화 적합성**: 로그 파일 단위 분석은 작업 간 의존성이 없어 임의의 개수로 분할/병렬화가 가능하다. 이는 실행 단위 수 변화에 따른 성능 확장성을 체계적으로 실험하기에 이상적인 구조이다.
- **구조적 비교 가능성**: 동일한 로그 분석 작업을 child process방식과 thread 방식으로 각각 구현할 수 있어, 메모리 공유 여부, 생성 비용, 컨텍스트 스위칭 오버헤드 등 두 실행 단위의 구조적 차이를 동일한 조건에서 비교할 수 있다. 또한 각 방식에 최적화된 구현을 서로 교차 적용함으로써 성능 차이의 원인을 구조 수준에서 분석할 수 있다.
- **동기화 문제 재현 가능성**: 최적화된 구현에서는 worker별 처리 구간과 결과 저장 공간을 분리하여 race condition이 발생하지 않도록 설계할 수 있다. 반면 공유 변수에 대한 동시 접근 상황을 의도적으로 구성하면 race condition을 재현할 수 있으며, 이를 통해 임계 구역, mutex, atomic operation 등의 동기화 개념을 코드 수준에서 실험하고 해결 과정을 제시할 수 있다.

1.3 설계 목표

본 프로젝트의 최종 목표는 child process와 thread를 적절히 혼합하여 CPU 코어 수에 비례하는 성능 향상을 달성하는 최적 시스템을 도출하고, 해당 시스템에서 발생하는 동기화 문제를 해결하는 것이다. 이를 위해 순차 실행에서 출발하여 실험 가~마의 단계적 과정을 거치며 다음 다섯 가지 세부 목표를 달성한다.

- 순차 실행 대비 child process 병렬처리 성능 검증 (실험 가) parent process만 사용하는 순차 실행을 baseline으로 설정하고, child process를 도입하였을 때 성능이 얼마나 향상되는 wall time, speedup, efficiency 지표로 측정한다. 이를 통해 병렬처리 도입의 타당성을 정량적으로 확인한다.
- Child process와 thread의 구조적 차이 및 최적화 방향 도출 (실험 나) 단일 child process와 단일 thread를 각각 최적 성능이 나오는 방향으로 구현하고 비교한다. 이후 child process에 최적화된 코드를 thread로, thread에 최적화된 코드를 child process로 교차 구현하여 실행 단위의 구조적 특성이 성능에 미치는 영향을 분석한다.
- 다중 확장 시 선형 성능 향상 달성 및 최적화 (실험 다) 단일 실행 단위를 다중으로 확장할 때 worker 수에 비례한 성능 향상이 나타나는지 검증한다. 이상적 선형 확장에 미치지 못하는 경우 스케줄링 오버헤드, 컨텍스트 스위칭, 메모리 경합 등 원인을

규명하고 구조를 개선하여 최적점을 도출한다.

- Child process와 thread혼합 시스템의 전체 최적화 (실험 라) 실험 다의 결과를 바탕으로, child process에 적합한 동작과 thread에 적합한 동작을 구분하여 두 실행 단위를 혼합한 시스템을 설계한다. 이를 통해 시스템 전체 차원에서 각 실행 단위의 장점을 극대화하는 구성을 도출하고, 이것이 최적임을 실험 데이터로 증명한다.
- 동기화 문제 유도.해결 및 성능 영향 분석 (실험 마) 실험 라의 최적 시스템에서 race condition이 발생하는 상황을 의도적으로 유도하고, 동기화를 사용하지 않았을 때 발생하는 문제를 확인한다. 이후 mutex 또는 thread-local result 방식으로 문제를 해결하고, 동기화 적용 전후의 정확성과 성능 차이를 비교한다.

위 목표를 종합하면, 본 프로젝트는 단순한 병렬처리 구현이 아니라 (1) 순차 -> 병렬 전환의 성능 향상 검증, (2) child process와 thread의 구조적 차이 분석, (3) 다중 확장 시 선형 성능 달성 및 최적화, (4) 혼합 구조를 통한 시스템 전체 최적화, (5) 동기화 문제 해결이라는 다섯 단계에 목표를 순차적으로 달성하는 것을 지향한다.

1.4 보고서 구성

본 보고서는 문제 정의에서 출발하여 단계적 실험과 분석을 거쳐 최적 시스템 도출에 이르는 과정이 논리적으로 이어지도록 다음과 같이 구성되어 있다. 2장 시스템 개요에서는 HMM 기반 로그 분석 시스템의 전체 동작 흐름과 구성요소를 설명하고, 각 구성요소의 병렬화 가능성을 분석한다. 아울러 child process와 thread의 적용기준, IPC 설계, 동기화가 필요한 구간을 식별하고, 최종 프로그램 구성 순서도를 제시한다.

3장 프로그램 구성에서는 개발 환경, 전체 코드 아키텍처, parent process와 child.thread 각각의 역할과 구현 방식, IPC 및 동기화 메커니즘을 상세히 기술한다. 단일에서 다중으로 확장되는 각 구조의 설계 변화도 함께 설명한다.

4장 동작 검증 및 성능 분석에서는 실험 설계와 test vector 선정 근거를 제시한 뒤, 실험 가~마를 순서대로 수행한 결과를 분석한다. 순차 실행 대비 child process 도입 효과(가), child process와 thread의 단일 및 교차 비교(나), 다중 확장 성능 및 최적화 과정(다), 혼합 시스템 최적 구성 도출(라), 동기화 유무 비교(마)를 단계적으로 다루며, 구현 과정에서 발생한 문제와 해결 과정도 함께 기술한다.

5장 결론에서는 주요 성과를 요약하고, 최적 시스템 구성의 의미와 현재 구현의 한계 및 향후 확장 방향을 제시한다.

6장 팀 운영에서는 팀원별 역할 분담, 기여도 자체 평가, 수행 계획서, 프로젝트 수행 일지를 정리하여 제시한다.

2. 시스템 개요

2.1 시스템 구성 및 동작 설명

2.1.1 선택한 시스템의 역할과 기능

본 프로젝트에서 구현한 시스템은 HMM(Hidden Markov Model) 기반 대량 로그 분석 시스템이다. 이 시스템은 다수의 로그 파일을 입력받아 각 로그에 대해 분석을 수행하고, 전체 결과를 집계하여 악성코드 탐지 여부와 성능 지표를 출력한다.

- 입력 : HMM 모델 디렉토리, 로그 파일 목록, Viterbi 반복 계수(R), External 반복 계수(EXT_R)
- 출력 : 로그별 악성코드 분류 결과(benign/ransomeware/trojan/spyware/rootkit), 전체 탐지 성공률, 실행 시간 및 성능 지표

시스템 수행하는 핵심 연산은 다음 두가지이다.

Viterbi 연산은 HMM 모델과 로그 관측열을 입력받아 가장 가능성이 높은 상태열을 추론하는 연산이다. 상태 전이 확률과 관측 확률을 기반으로 동적 프로그래밍을 반복 수행하며, 반복 계수 R에 비례하여 연산량이 결정된다. 각 로그에 대한 Viterbi 연산은 다른 로그의 결과에 전혀 의존하지 않아 완전한 독립성을 가진다.

External 연산은 Viterbi 분석 결과를 바탕으로 외부 모델(external model)을 누적 갱신하고, 갱신된 모델을 통해 악성코드 계열을 최종 판별하는 연산이다. External 반복 계수 EXT_R에 비례하여 연산량이 결정되며, 연산 과정에서 external model에 대한 쓰기 접근이 반복적으로 발생한다. External 연산의 도입 배경은 2.2.2절에서 상세히 기술한다.

2.1.2. 시스템 구성요소 분류

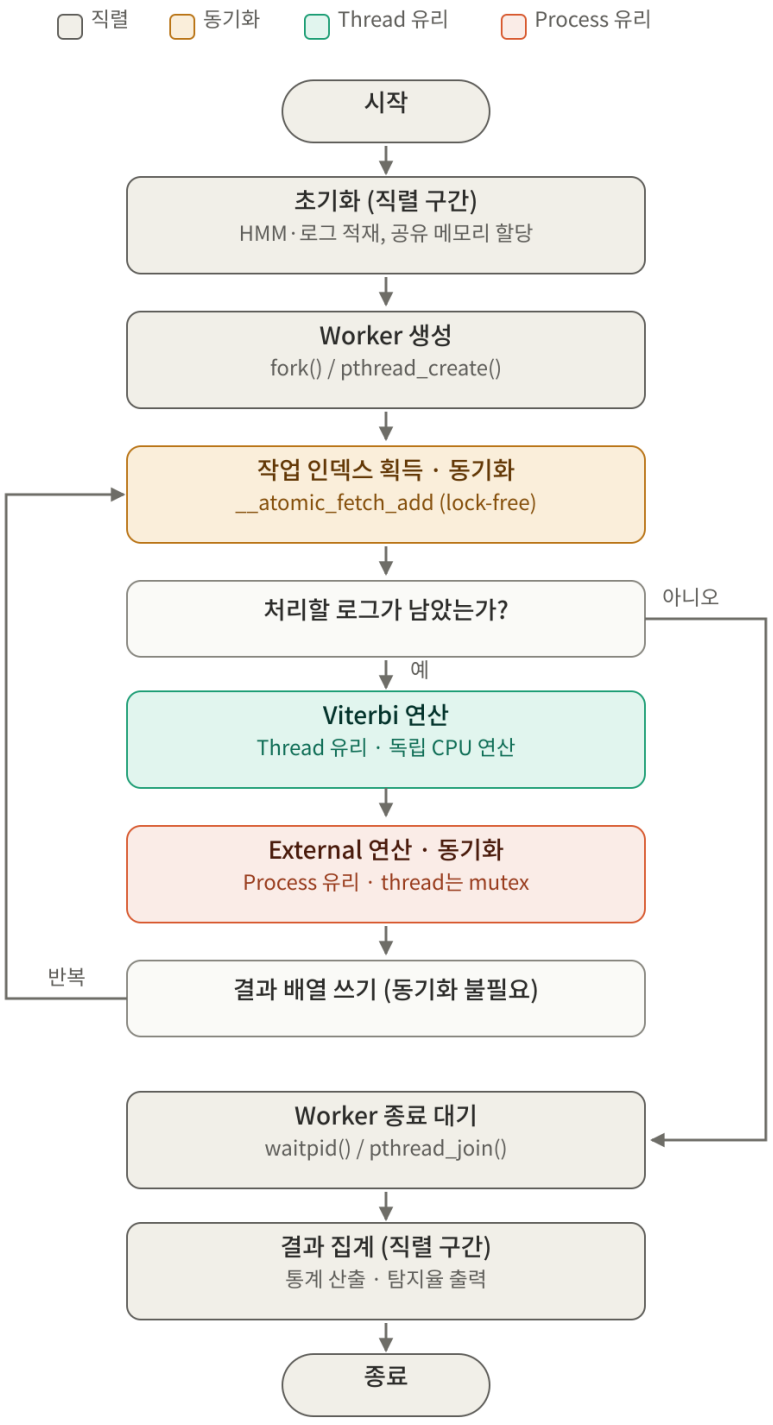
시스템의 전체 동작을 기능별로 분류하면 다음 다섯 가지 구성요소로 나눌 수 있다.

1. 초기화: HMM 모델 로드, 로그 파일 목록 파싱, 로그 데이터 전체 메모리 적재, 결과 배열 할당, 시스템 시작 시 한번만 수행되며, 이후 모든 worker가 공통으로 사용하는 데이터를 준비한다.
2. 작업 분배: 전체 로그를 worker에게 할당하는 단계. 실험 구성에 따라 static range 또는 round-robin 방식을 사용한다.
3. Viterbi 연산: 각 worker가 배정받은 로그에 대해 HMM 상태열 추론을 수행하는 단계. 로그 단위로 독립적으로 수행된다.
4. External 연산: Viterbi 결과를 바탕으로 external model을 갱신하고 악성코드 계열을

판별하는 단계. 연산 결과를 model에 반영하는 과정에서 공유 상태에 대한 쓰기 접근이 발생한다.

5. 결과 집계: 모든 worker의 분석 결과를 수집하고 전체 통계를 산출하는 단계. 모든 worker가 작업을 완료한 후 단일 흐름으로 수행된다.

2.1.3 전체 동작 순서도



2.2 문제 분석

2.2.1 병렬처리 가능 구성요소 파악

각 구성요소의 병렬화 가능 여부와 특성을 분석하면 다음과 같다. 초기화는 병렬화 대상이 아니다. 이후 모든 worker가 공통으로 사용할 데이터를 준비하는 단계이므로 worker 생성 이전에 단일 흐름으로 완료되어야 한다. 실험 결과 로그 파싱 포함 약 1.3초가 소요되어 전체 실행 시간(순차 기준 약 83) 대비 약 1.6%에 불과하다.

작업 분배는 경량 연산으로 병렬화 대상에 포함되지 않는다. Viterbi 연산은 병렬화 효과가 가장 큰 구성요소이다. 각 로그에 대한 Viterbi 연산은 다른 로그의 결과에 전혀 의존하지 않아 완전한 독립성이 보장된다. 연산 중 I/O가 발생하지 않으며 CPU 사용률이 약 99%수준으로 유지되는 순수한 CPU-bound 연산이다. External 연산은 연산 자체는 CPU-bound이지만, 연산 결과를 external model에 반영하는 과정에서 공유 상태에 대한 쓰기 접근이 발생한다. 이 특성이 process와 thread의 구조적 차이를 가장 명확하게 드러내는 지점이며, 2.2.2절에서 상세히 분석한다. 결과 집계는 모든 worker 완료 후에 수행되어야 하므로 병렬화 대상이 아니다.

결론적으로, Viterbi 연산과 external 연산이 전체 실행 시간의 약 97% 이상을 차지하며 이 두 연산이 병렬처리의 핵심 대상이다.

2.2.2 External 연산 도입 배경 – 예비 실험을 통한 설계 문적 인식

본 시스템의 초기 설계에서는 Viterbi 연산만으로 병렬처리 성능을 비교하는 방안을 검토하였다. 그러나 이 구성에서는 process와 thread간의 공정한 비교가 불가능하다는 문제가 실험을 통해 확인되었다.

Viterbi 단독 구성에서는 thread는 child process 대비 약 21.8% 빠른 성능을 보였다. 이 결과의 원인을 구조적으로 분석하면 다음과 같다.

Viterbi 연산은 각 로그가 독립적으로 처리되므로, process와 thread 모두 worker별 독립실행이 가능하다. 그런데 이 조건에서는 thread는 HMM 모델과 로그 배열을 공유 주소 공간에서 별도의 복사 없이 직접 접근할 수 있는 반면, child process는 동일한 데이터에 접근하기 위해 fork() 후 copy-on-write 방식을 거치거나 shared memory를 통해야 한다. 즉 Viterbi 단독 구성에서는 thread가 공유 주소 공간을 통해 데이터 접근 비용을 절감하는 구조적 이점을 일방적으로 가지며, process가 이점을 가지는 조건이 존재하지 않는다.

이는 Viterbi 단독 구성으로는 '각 실행 단위가 가장 유리한 조건에서 최선의 성능을

낼 수 있도록 설계하여 비교하라'는 실험 나번의 요구 조건을 충족할 수 없음을 의미한다. Process가 구조적 이점을 가질 수 있는 연산이 시스템 내에 존재해야 한다.

이 문제를 해소하기 위해 external 연산을 도입하였다. External 연산은 연산 결과를 external model에 누적 반영하는 과정에서 공유 상태에 대한 쓰기 접근이 반복적으로 발생한다. 이 특성이 process와 thread에 각각 다른 방식으로 작용한다.

- **Process의 구조적 이점** : 각 process는 독립된 주소 공간을 가지므로 external model을 process 내부에 local로 유지할 수 있다. 이 경우 model 갱신이 전적으로 해당 process의 주소 공간 내에서 이루어지므로 다른 process와의 동기화가 불필요하다. Process 독립 주소 공간이 lock 없는 상태 관리라는 구체적 이점으로 작동한다.
- **Thread 구조적 이점**: Thread는 HMM 모델, 로그 배열, 결과 배열, 작업 인덱스를 공유 주소 공간에서 직접 접근할 수 있어 데이터 복사와 IPC 비용이 발생하지 않는다. 단 shared model 갱신 시에는 mutex 등의 동기화가 필요하다.

External 연산의 도입으로 인해 process와 thread 모두 작업 특성에 따른 구고적 이점을 가지게 되며, 나번 실험에서 각 실행 단위가 유리한 조건과 불리한 조건을 모두 실험적으로 검증할 수 있다.

2.2.3 병목 구간 및 병렬화 제약 분석

병렬화를 제약하는 요인은 다음 세 가지이다.

- **직렬 구간(초기화)** : 로그 파싱 및 메모리 적재는 worker 생성 이전에 단일 흐름으로 수행된다. 전체 실행 시간 대비 약 1.6%에 해당하므로 Amdahl의 법칙에 따른 이론적 병렬화 효율 상한은 약 98.4% 수준이다.
- **Worker 생성 및 회수 overhead** : fork()와 waitpid(), 또는 pthread_create()와 pthread_join()에 소요되는 시간이 발생한다. 실험 결과 fork 생성은 약 0.35ms, shared memory 초기화는 약 0.05ms로 측정되어 전체 실행 시간 대비 무시 가능한 수준이다.
- **메모리 대역폭 경쟁** : worker 수가 증가할수록 다수의 실행 단위가 동시에 메모리에 접근하면서 메모리 대역폭 경쟁이 발생할 수 있다. 실험 가에서 worker 5개 구성 시 efficiency가 0.880으로 감소한 원인 중 하나로 분석된다.

2.3 문제 해결 방향

2.3.1 Process-Thread 대응 설계

2.2절의 분석을 바탕으로, 각 실행 단위의 구조적 이점이 극대화되는 작업 배정 기준을 다음과 같이 도출하였다.

작업	적합한 실행 단위	근거
Viterbi 연산	Thread	대규모 read-only 데이터를 복사 없이 직접 공유 가능, 결과 배열도 인덱스별 분리로 동기화 불필요
External 연산	Process	독립 주소 공간에서 local external-model을 운용하므로 lock 없이 안전한 상태 관리 가능

이 기준은 나번 실험에서 B1-B4 설계의 근거가 되며, 라번 혼합 시스템 설계에서 최종적으로 적용된다.

2.3.2 IPC(프로세스 간 통신)설계

Process 기반 병렬처리에서는 실행 단위들이 독립된 주소 공간을 가지므로 데이터 공유를 위한 IPC가 필요하다. 본 시스템에서 사용하는 IPC 메커니즘은 다음과 같다.

- **익명 공유 메모리(mmap, MAP_SHARED | MAP_ANONYMOUS)** : worker 간 작업 인덱스, 결과 배열, worker별 통계를 공유하기 위해 사용된다. fork() 이전에 공유 메모리 영역을 생성하여 parent와 child 모두 동일한 물리 메모리에 접근하도록 구성한다.
- **원자적 작업 인덱스 관리**: 작업 인덱스는 __atomic_featch_add 를 통한 원자적 증가 연산으로 처리하여 mutex 없이 worker 간 작업 분배를 안전하게 수행한다.
- **waitpid를 통한 종료 동기화**: parent process는 자신의 작업 완료 후 waitpid로 child process들의 종료를 대기하고, 이후 공유 메모리에서 결과를 집계한다.

IPC overhead를 최소화하기 위해 shared memory 초기화는 fork 이전에 완료하고, worker 실행 중 IPC를 통한 초기화는 atomic 작업 인덱스 접근을 제외하면 발생하지 않도록 설계하였다.

2.3.3 동기화(Synchronization) 필요 구간 식별

시스템 내에서 동기화가 필요한 구간은 다음과 같다.

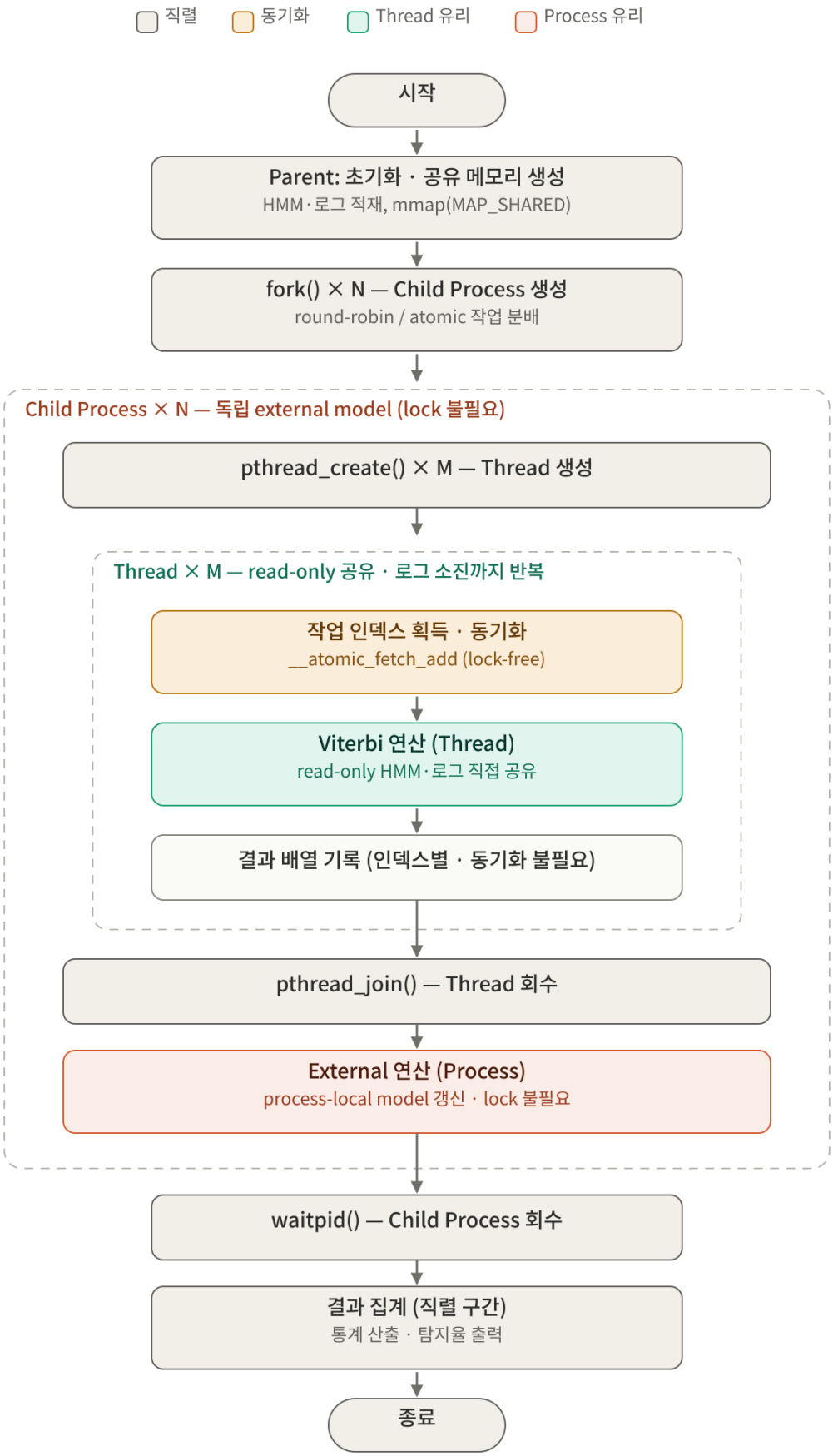
- **작업 인덱스 접근**: 여러 worker가 공유 작업 큐에서 다음 처리할 로그 인덱스를 가져올 때 race condition이 발생할 수 있다. __atomic_fetch_add를 통한 원자적 증가 연산으로 처리하여 mutex 없이 해결하였다.
- **External model 갱신(thread 방식)** : thread 방식에서 shared external model에 여러 thread가 동시에 쓰기 접근할 경우 race condition이 발생한다. 이 구간은 마번 실험에서 동기화 유무에 따른 정확성과 성능 차이를 비교하는 핵심 임계 구역이다. 최적화된 구현에서는 chunk 단위 commit 시 mutex lock을 적용하여 이 구간을 보호한다.
- **결과 배열 쓰기**: 각 로그의 분석 결과는 해당 로그 인덱스의 결과 슬롯에만 기록되므로, 서로 다른 worker가 동일한 슬롯에 접근하는 일이 발생하지 않는다. 따라서 결과 배열 쓰기는 동기화가 불필요하다.
- **Worker 종료 대기**: process 방식에서는 waitpid, thread 방식에서는 pthread_join을 통해 모든 worker의 완료를 보장한 후 결과 집계를 시작한다.
-

2.4 최종 프로그램 구성 및 순서도

위 분석을 종합하면, 본 프로젝트의 실험은 순차 실행에서 출발하여 단계적으로 병렬 구조를 발전시키는 흐름으로 구성된다. 각 실험 단계의 구성과 핵심 변경 사항은 다음과 같다.

실험	실행 단위 구성	핵심 변경 사항
가	Parent process(순차) -> parent + child process(병렬)	Child process 도입, round-robin 분배
나	단일 child process/ 단일 thread 교차 비교	External 도입, work-conserving 구조, B1-B4 설계
다	다중 child process / 다중 thread 확장	Worker 수 증가에 따른 확장성 분석 및 최적화
라	다중 child process + 다중 thread 혼합	작업 특성별 실행 단위 최적 배정
마	라의 최적 시스템에 동기화 적용 / 미적용 비교	Race condition 유도 및 해결

동기화가 필요한 구간(extennal model 갱신, 작업 인덱스 접근)은 최종 순서도에서 명시적으로 표시되며, 마번 실험에서 동기화를 의도적으로 제거한 조건과 비교된다.


직렬
동기화
Thread 유리
Process 유리
시작
Parent: 초기화 · 공유 메모리 생성
HMM·로그 적재, mmap(MAP_SHARED)
fork() × N — Child Process 생성
round-robin / atomic 작업 분배
Child Process × N — 독립 external model (lock 불필요)
pthread_create() × M — Thread 생성
Thread × M — read-only 공유 · 로그 소진까지 반복
작업 인덱스 획득 · 동기화
__atomic_fetch_add (lock-free)
Viterbi 연산 (Thread)
read-only HMM·로그 직접 공유
결과 배열 기록 (인덱스별 · 동기화 불필요)
pthread_join() — Thread 회수
External 연산 (Process)
process-local model 갱신 · lock 불필요
waitpid() — Child Process 회수
결과 집계 (직렬 구간)
통계 산출 · 탐지율 출력
종료

3. 프로그램 구성

3.1 개발 환경 및 프로그램 구성

실험이 진행된 대상 시스템의 하드웨어 스팩은 단일 소켓 내에 독립적인 5개의 물리 코어를 탑재하고 있다. 이와 같은 격리된 멀티코어 환경은 운영체제의 스케줄링 변수를 최소화하고 실행 단위의 추가에 따른 성능 확장성을 왜곡 없이 측정하기에 적합하다.

소프트웨어 개발 스택 환경으로는 리눅스(Linux) 운영체제 위에서 표준 C/C++ 언어를 활용하여 프로그래밍 되었으며, GNU 컴파일러 모음(GCC)을 통해 빌드되었다. 저수준 자원 제어와 병렬 처리를 구현하기 위해 외부 라이브러리에 의존하지 않고 POSIX 표준 시스템 콜 및 API를 직접 호출하도록 소스 코드가 작성되었다. 이에 따라 다중 프로세스 구현에는 fork()와 waitpid()가 사용되었고, 다중 스레드 구현체에는 pthread_create() 및 pthread_join() 라이브러리가 적용되었다. 프로세스 간 통신(IPC)을 위한 커널 공간 매핑에는 가상 메모리 시스템 콜인 mmap()을 활용하여 MAP_SHARED | MAP_ANONYMOUS 플래그 조합으로 익명 공유 메모리 영역을 확보했으며, 동기화가 필요한 전역 인덱스 제어에는 GCC 내장 함수인 __atomic_fetch_add()를 호출하여 하드웨어 수준의 원자적 연산을 수행하도록 코드를 구성하였다.

전체 프로그램 아키텍처는 코드의 재사용성을 높이고 실험 간의 공정한 대조 환경을 제공하기 위해 하나의 단일 메인 진입점(Entry Point) 함수를 공유하도록 소스 코드가 통합 구현되었다. 프로그램 실행 시 입력되는 인자(Argument) 조합에 따라, 순차 실행 모드, 라운드 로빈 기반 다중 프로세스 모드, Work-Conserving 기반 단일/다중 실행 모드, 그리고 최종 프로세스-스레드 혼합형(Hybrid) 모드가 동적으로 선택되어 분기 실행된다. 모든 모드는 실행 직후 HMM 모델 디렉토리와 로그 리스트를 파싱하여 메인 힙 영역에 적재하는 직렬 초기화 과정을 공통으로 거치며, 병렬화 모드 구동 시에는 커널 락(Lock) 경합 및 불완전한 상태 복사 문제를 원천 차단하기 위해 공유 메모리 할당 및 fork()를 통한 자식 프로세스 생성이 스레드 생성 단계보다 반드시 선행되도록 구현 순서를 엄격히 제어하였다.

4. 동작 검증 및 성능 분석

4.1 실험 설계

4.1.1 성능 측정 지표 및 측정 방법

모든 실험에서 공통으로 사용하는 성능 지표는 다음과 같다. Wall time은 프로그램 시작부터 종료까지의 실제 경과 시간(초)으로, 사용자가 체감하는 처리 시간이다. Speedup은 순차 실행 wall time 대비 병렬 실행 wall time의 비율(Speedup = T_sequential / T_parallel)로, 병렬처리 도입의 효과를 정량화한다. Efficiency는 Speedup을 worker 수로 나눈 값(Efficiency = Speedup / N)으로, 각 실행 단위가 얼마나 효율적으로 활용되는지를 나타낸다. 이상적인 경우 1.0이며, 1.0 미만이면 오버헤드 또는 load imbalance가 존재함을 의미한다. CPU 사용률은 worker별 CPU 점유율로, 연산이 실제로 병렬 수행되고 있는지 검증하는 보조 지표이다.

측정 신뢰도 확보를 위해 모두 실험은 CPU affinity를 고정하여(cpu=0 기준) 스케줄링 변동을 최소화하였고, 결과의 정확성은 Viterbi checksum, external checksum, external score sum의 일치 여부로 검증하였다.

4.1.2 Test Vector 설계

모든 실험에 공통으로 적용한 test vector는 다음과 같다. 총 로그 수는 10,000개 이며, 총 API 호출 수는 7,695,438회이다. 시퀀스 길이는 최소 400, 평균 769.54, 최대 1,798로 분포한다. Viterbi 반복 계수(R)는 100, External 반복 계수(EXT_R)는 500이다. 순차 실행 기준 wall time은 89.732 s로, 이 값을 Speedup 산출의 baseline으로 사용한다.

4.2 실험 가. 순차 실행 vs Child Process 병렬처리

4.2.1 실험 조건

실험 가는 child process를 생성하지 않고 parent process 단독으로 모든 로그를 순차 처리하는 기준(baseline)과, parent 1개 + child N개(N = 1, 2, 3, 4)로 구성된 총 (N+1)개의 worker가 로그를 병렬 처리하는 구성의 성능을 비교한다.

작업 분배는 round-robin 방식을 채택하여 각 worker가 로그 인덱스 기준으로 균등하게 연산을 분담하도록 하였으며, 모든 worker는 Viterbi와 External 연산 전체를 독립적으로 수행한다. 병렬화 효율 측정의 기준점인 baseline wall time은 순차 실행 측정값인 89.732 s로 설정하였다.

4.2.2 실험 결과

순차 실행(가-0)에서는 단일 CPU(cpu=0)에 고정된 parent process 1개가 10,000개 로그 전체에 대해 Viterbi + External 연산을 순차적으로 수행하였다. 전체 실행 시간 89.731 s, CPU 사용률 99.67%, 악성코드 탐지 성공률 99.80%(9,980 / 10,000)를 기록하였다. 단계별 timing에서 Sequential Viterbi+External loop가 전체 시간의 99.56%를 차지하여, 이 구간이 병렬화의 핵심 대상임을 확인하였다.

병렬 실행 실험(가-1~4)의 측정 결과는 아래의 표와 같다.

구성	Worker 수	Wall time (s)	CPU 사용률	Speedup	Efficiency	Load Imbalance
순차 (baseline)	1 (parent)	89.731	99.67%	1.000	1.000	-
Child 1 (총 2)	2	42.512	194.55%	2.111	1.055	1.005
Child 2 (총 3)	3	27.999	294.62%	3.205	1.068	1.013
Child 3 (총 4)	4	21.134	391.65%	4.246	1.061	1.015
Child 4 (총 5)	5	17.990	458.96%	4.988	0.995	1.035

모든 병렬 실험에서 악성코드 탐지 성공률은 99.80%로 동일하게 유지되었으며, viterbi checksum, external checksum, external score sum이 순차 실행 결과와 완전히 일치하여 결과 정확성을 검증하였다. waitpid abnormal count는 전 구성에서 0이었다.

4.2.3 분석 및 결론

첫째, child process 도입은 즉각적이고 유의미한 성능 향상을 제공하였으며, worker 2~4 구성에서 Efficiency가 1.0을 초과하는 초선형(superlinear) 성능 향상이 관찰되었다. child 1개(총 worker 2) 추가 시 wall time이 약 53% 감소하고 Speedup 2.111을 기록하였다. worker 3에서 Efficiency 1.068로 최댓값을 보였으며 worker 4에서도 1.061을 유지하였다.

이 초선형 현상의 구조적 원인은 CPU 캐시 계층 효과이다. 순차 실행 시 단일 CPU가 10,000개 로그를 처리할 때 작업 집합이 L1/L2 캐시 용량을 초과할 수 있다. 병렬 실행 시 각 worker가 처리하는 로그 수가 줄어들면서 작업 집합이 캐시에 수용 가능한 크기로 축소되고, 캐시 적중률 향상으로 인해 총 CPU time이 감소한다. 실제로 병렬 실험 전체에서 총 CPU time 합이 82.5~82.8 s 범위로 순차 실행(89.4 s) 대비 약 7~8% 감소하였으며, 이는 Amdahl의 법칙이 가정하는 총 연산량 불변 전제가 캐시 효과로 인해 위반된 사례에 해당한다.

둘째, worker 수가 물리 코어 수(5개)에 근접하면서 Efficiency가 하락하는 경향이 나타났다. child 4개(총 5 worker) 구성에서 Efficiency가 0.995로 1.0 미만으로 떨어졌으며, load imbalance ratio도 1.035로 소폭 증가하였다. 이는 worker 수가 코어 수에 근접함에 따라 메모리 대역폭 경쟁과 OS 스케줄링 오버헤드가 누적되는 데 기인한다.

셋째, fork() 생성 및 shared memory 초기화 오버헤드는 전체 실행 시간 대비 무시 가능한 수준이었다. 단계별 timing에서 fork creation은 0.28~1.04 ms, shared memory mmap + init은 0.02~0.05 ms로 측정되어 전체의 0.01% 이하에 해당한다.

종합하면, child process 기반 병렬처리는 순차 실행 대비 명확한 성능 향상을 제공하며, round-robin 분배 방식이 worker 간 부하를 균등하게 분산시키는 데 효과적임을 확인하였다. worker 수가 물리 코어 수에 근접할수록 Efficiency가 감소하는 경향은 이후 실험(다, 라)에서 최적 worker 수를 결정하는 기준으로 활용된다.

4.3 실험 나. 단일 Child Process vs 단일 Thread

4.3.1 각 방식에 최적화된 코드 구성

실험 나는 단일 child process와 단일 thread를 각각 최적 성능이 나오는 방향으로 구현하고, 교차 비교를 통해 실행 단위의 구조적 특성이 성능에 미치는 영향을 분석하는 실험이다. 2.2.2절에서 분석한 바와 같이, Viterbi 연산은 read-only 데이터에 대한 독립 연산으로 thread의 공유 주소 공간이 유리하고, External 연산은 모델 갱신을 수반하므로 process의 독립 주소 공간이 유리하다고 가정하였다.

이 가정을 바탕으로 child process에 최적화된 설계(B1)와 thread에 최적화된 설계(B3)를 각각 구현하였으며, 교차 비교를 위해 B2와 B4를 추가로 구현하였다. 모든 B 실험은

공통적으로 work-conserving 구조를 채택한다. 이 구조에서 main(parent)의 주요 역할은 Viterbi, child의 주요 역할은 External이지만, 자신의 queue가 소진되면 반대 역할의 queue에서 작업을 가져와(work stealing) 수행한다. 실행 구성은 모두 main 1 + child 1 = 총 worker 2이며, 각 worker는 별도 CPU에 고정(main=cpu0, child=cpu1)하였다.

네 가지 실험 구성의 설계 특성은 다음과 같다.

B1 (Child Process-Friendly: Local External Model): process의 독립 주소 공간을 최대한 활용하는 설계이다. External model을 각 process의 로컬 메모리에 유지하여 model 갱신 시 어떠한 동기화도 필요하지 않다. process 간 데이터 공유는 shared memory(mmap)의 atomic 작업 인덱스와 결과 배열에 한정하여 IPC 비용을 최소화하였다.

B2 (Thread + Chunk Shared Model Lock): thread 환경에서 shared external model을 운용하되, 동기화 비용을 최소화하기 위해 chunk 단위 commit 방식을 적용한 설계이다. 각 thread는 External 결과를 로컬에 축적하다가 일정 크기(chunk commit event size = 16,384, shared updates per commit = 8)에 도달하면 mutex lock을 획득하여 shared model에 일괄 반영한다.

B3 (Thread-Friendly: 역할 분리 + 직접 공유): thread의 공유 주소 공간을 최대한 활용하는 설계이다. HMM 모델, 로그 배열, 결과 배열, 작업 인덱스를 thread 주소 공간에서 직접 공유하여 데이터 복사와 IPC 비용을 완전히 제거한다.

B4 (교차 비교: Thread-Friendly 설계의 Process 구현): B3와 동일한 역할 분리 및 work-conserving 로직을 process + shared memory(mmap)로 구현한 교차 비교 실험이다. thread에 최적화된 설계가 process 환경에서 어떤 성능 변화를 보이는지 확인하기 위한 대조군이다.

4.3.2 실험 결과

네 가지 구성의 측정 결과는 아래의 표와 같다.

구성	실행 단위	설계 최적화 대상	Span (s)	CPU time 합 (s)	CPU 사용률	
B1	Process	Process (local model)	59.004	117.861	196.71 %	lock 없음
B2	Thread	Thread (chunk lock)	60.184	120.242	196.92 %	lock 239,921회, wait 0.015

구성	실행 단위	설계 최적화 대상	Span (s)	CPU time 합 (s)	CPU 사용률	
						s
B3	Thread	Thread (직접 공유)	60.976	121.883	198.39 %	직접 공유
B4	Process	Thread 설계 교차 적용	82.513	164.931	197.80 %	B3 로직을 Process 구현

아래의 표는 각 구성에서 worker별 작업 분배 현황을 보여준다.

구성	Worker	Role	Viterbi 수	External 수	Wall (s)	CPU%	
B1	parent	vit-first	10,000	1,891	59.004	99.90	steal 1,891
B1	child	ext-first	0	8,109	59.003	99.86	primary 8,109
B2	main	vit-first	10,000	2,002	60.180	99.88	steal 2,002
B2	child	ext-first	0	7,998	60.184	99.92	primary 7,998
B3	main	vit-first	10,000	1,862	60.976	99.97	steal 1,862
B3	child	ext-first	0	8,138	60.975	99.92	primary 8,138
B4	parent	vit-first	10,000	2,333	82.513	99.94	steal 2,333
B4	child	ext-first	0	7,667	82.512	99.94	primary 7,667

4.3.3 분석 및 결론

첫째, Process-Friendly 설계(B1)가 가장 우수한 성능을 달성하였다. B1은 span 59.004 s로, B3(60.976 s) 대비 3.2%, B2(60.184 s) 대비 2.0% 빠른 성능을 보였다. 이는 process의 독립 주소 공간이 external model 갱신에서 구조적 이점을 제공한다는 2.2.2절의 설계 가설을 실험적으로 확인하는 결과이다. B1의 총 CPU time(117.861 s)이 B3(121.883 s) 대비 3.3% 적은 것은, local external model이 해당 process의 캐시에 상주하여 캐시 적중률이 높게 유지되고 다른 worker와의 캐시 일관성(cache coherence) 트래픽이 발생하지 않기 때문이다.

둘째, Thread-Friendly 설계를 Process로 교차 구현한 B4는 극단적인 성능 저하를 보였다. B4는 span 82.513 s로, 동일 로직의 thread 구현인 B3(60.976 s) 대비 35.3% 느리고, process 최적 설계인 B1(59.004 s) 대비 39.8% 느린 결과를 기록하였다. B4의 총 CPU time(164.931 s)은 B1(117.861 s) 대비 39.9% 높아, 동일한 연산을 수행하는 데 실질적으로 더 많은 CPU cycle이 소비되었음을 보여준다. 이는 thread-friendly 설계가 전제하는 공유 주소 공간에서의 저비용 직접 접근이 process + shared memory 환경에서는 성립하지 않기 때문이다. 이 결과는 코드의 메모리 접근 패턴이 실행 단위의 메모리 구조와 반드시 일치해야 한다는 원칙을 실증적으로 보여준다.

셋째, B2의 chunk lock 방식은 동기화 비용을 효과적으로 억제하였다. B2는 span 60.184 s로, B3(60.976 s)보다 오히려 0.792 s(1.3%) 빠른 결과를 보였다. 총 lock 획득 횟수는 239,921회였으나 총 lock 대기 시간은 0.015 s에 불과하여, chunk 단위 commit이 lock 경합 빈도를 충분히 낮춘 덕분이다. 또한 B3에서는 매 External 연산마다 shared 영역에 직접 쓰기가 발생하여 캐시 무효화(cache invalidation)가 빈번한 반면, B2에서는 로컬에 결과를 축적한 후 일괄 commit하므로 shared 영역에 대한 쓰기 빈도가 대폭 감소한다. 이 결과는 실험 마에서 chunk lock 기반 동기화가 실용적 해법이 될 수 있음을 시사한다.

넷째, work-conserving 구조는 전 구성에서 완벽한 부하 균등을 달성하였다. 모든 구성에서 두 worker의 wall time 차이가 0.001~0.004 s 이내로, work stealing 메커니즘이 Viterbi와 External 간 연산량 불균형을 동적으로 해소하여 유휴 시간 없이 CPU를 최대한 활용함을 확인하였다.

다섯째, 실험 나의 결과는 실행 단위 선택과 코드 설계가 불가분의 관계임을 보여준다. B1(process-friendly + process)과 B3(thread-friendly + thread)가 각각 59.004 s, 60.976 s로 유사한 수준의 성능을 보인 반면, B4(thread-friendly + process)는 82.513 s로 극단적인 저하를 기록하였다. process는 독립 주소 공간에서 local 상태를 운용할 때 최적이며, thread는 공유 주소 공간에서 대규모 read-only 데이터를 직접 참조할 때 최적이다. 이 원칙은 실험 라에서 Viterbi 연산은 thread에, External 연산은 process에 배정하는 혼합 시스템 설계의 직접적 근거로 활용된다.

4.4 실험 다. 다중 Child Process / 다중 Thread 확장 결과

실험 다는 worker 수를 1개에서 5개까지 단계적으로 확장하면서 child process 방식과 thread 방식 각각의 성능 확장성을 비교한다. worker=1 구간은 실험 가-0의 순차 실행 결과(89.732 s)를 baseline으로 사용한다. worker=2 구간은 실험 나에서 측정한 단일 구현체(process는 B4, thread는 B3)의 실측값을 적용하여 1개부터 5개까지의 확장 추이를 연속적으로 분석한다.

4.4.1 실험 환경 및 조건

- **하드웨어 실행 환경**: 물리 코어 5 개 (Core(s) per socket = 5, Thread(s) per core = 1)
- **연산 작업량**: 파싱 대상 전체 로그 수 = 10,000 개, Viterbi 반복 계수 R = 100, External 반복 계수 EXT_R = 500
- **성능 기준점(Baseline):** 단일 순차 실행 시간(가-0 결과) = 89.732 초

4.4.2 실제 성능 비교 및 정량 분석

아래의 표는 실험 다 중 Child Porcess 기반 worker 확장 성능 분석을 위해 나타낸 결과이다.

worker 수	전체 실행 시간 (s)	Speedup (vs 순차)	Efficiency	CPU 사용률 (%)	최대 RSS (parent)
1	89.732	1.000	1.000	99.67%	34,200 KB 내외
2	83.509	1.075	0.537	197.80%	34,280 KB 내외
3	39.498	2.272	0.757	292.27%	34,308 KB
4	30.336	2.958	0.739	382.96%	34,340 KB
5	27.016	3.321	0.664	469.04%	34,372 KB

아래의 표는 실험 다 중 Thread 기반 worker 확장 성능 분석을 위해 나타내었다.

worker 수	전체 실행 시간 (s)	Speedup (vs 순차)	Efficiency	CPU 사용률 (%)	최대 RSS (전체)
1	89.732	1.000	1.000	99.67%	34,000 KB 내외
2	61.522	1.459	0.729	198.39%	34,800 KB 내외
3	38.855	2.309	0.770	296.59%	35,352 KB
4	29.392	3.053	0.763	394.61%	35,896 KB
5	24.724	3.629	0.726	468.05%	36,424 KB

worker 수가 5개로 확장됨에 따라 프로세스 버전은 27.016초(Speedup 3.321배), 스레드 버전은 24.724초(Speedup 3.629배)로 실행 시간이 단축되었으며, 전체 CPU 사용률은 각각 469.04%, 468.05%에 도달했다. 이는 시스템이 보유한 5개의 물리 코어를 거의 완전히 활용(Full Utilization)하고 있음을 증명한다. 또한 worker 간 연산 종료 wall time 편차가 0.01초 내외로 미미하여, 설계한 work-conserving 구조(Work Stealing)가 다중 확장 환경에서도 균등한 부하 분배(Load Balancing)를 성공적으로 수행함을 확인하였다.

코어를 충분히 점유함에도 worker=5 지점에서 이상적인 선형 확장(5배)에 완벽히 도달하지 못하는 원인은 작업 분배의 문제가 아니며, 다음과 같은 구조적 제약 요인으로 분석된다:

- **메모리 대역폭 경쟁 (Memory Bandwidth Contention)**: 본 연산은 대량의 로그와 HMM 모델을 빈번하게 참조하는 메모리 집약적(Memory-bound) 특성을 가지므로 실행 단위(worker)가 늘어남에 따라 공유 자원인 메모리 버스 대역폭의 경쟁이 심화되어 실행

파이프라인에 병목을 유발한다.

- **Amdahl 의 법칙에 따른 직렬 구간 존재**: 로그 파일 리스트 파싱 및 초기 메모리 적재 등은 병렬화되지 않고 단일 흐름으로 수행되는 고정 직렬 구간(순차 기준 전체의 약 1.6~2.6%)이다. 전체 병렬 연산 시간이 단축될수록 이 고정 직렬 구간이 차지하는 상대적 비중이 커지면서 전체 확장성의 상한선을 형성한다.

메모리 사용량(RSS) 측면에서는 주소 공간 분리(process)와 공유(thread)의 태생적 구조 차이가 측정값에 그대로 반영되었다. Process 방식은 자식이 독립 주소 공간을 가지므로 worker 수와 무관하게 RSS가 약 34,300 KB 선으로 균일하게 유지된 반면, Thread 방식은 프로세스 내의 힙 및 전역 메모리를 공유하더라도 스레드 수 증가에 따라 스택 영역 및 TCB 컨텍스트 할당 오버헤드가 누적되어 전체 RSS가 35,352 KB(worker=3)에서 36,424 KB(worker=5)로 누적 증가하는 특성을 보였다.

모든 worker 수 조건에서 Viterbi Checksum(24518236282129), 악성코드 탐지 성공률(99.80%), family_count 분포가 완벽히 동일하게 유지되었다. 이는 병렬 확장 및 동적 태스크 스틸링 환경 속에서도 Race Condition 없이 데이터 정합성이 완벽히 유지되고 있음을 증명한다.

4.4.3 성능 최적화 과정 및 임계점 검증

성능 저하의 원인이 부하 불균형이 아니라 메모리 대역폭 한계임을 검증하고 시스템의 실질적 한계 최적점을 도출하기 위해, 물리 코어 수(5개)를 초과하는 6~10개의 오버서브스크립션(Oversubscription) 구동 상태를 추가 검색하였다. 물리 코어 수를 초과한 구간에서는 worker 수를 더 늘려도 실행 시간이 단조 감소하지 않고 약 40~52초 범위 내에서 불안정하게 변동하는 거동을 표출하였다. 이는 하드웨어 레벨에서 추가적인 물리 연산 리소스를 얻지 못하므로, 메모리 접근 대기 시간을 다른 worker 연산으로 가릴 수 있는 지연 숨김(Latency Hiding) 이득보다 컨텍스트 스위칭 및 OS 스케줄링 오버헤드 증가라는 손해가 더 크게 작용하기 때문이다.

따라서 본 병렬 처리 시스템의 안정적인 최적 worker 수는 물리 코어 수와 일치하는 5개 부근이며, 무한정 실행 단위를 확장하는 구조는 성능 포화 및 오버헤드 유발만을 초래한다는 실질적 최적점을 증명하였다. 이 결과를 통해 후속 단계인 (라)번 혼합 시스템

아키텍처 설계에서 작업 성격별로 실행 단위를 배정하는 근거로서 사용한다.

4.5 실험 라. 다중 Child Process + 다중 Thread 혼합 시스템

4.5.1 실험 목적

실험 (라)는 parent process, child process, worker thread를 함께 사용하는 혼합 병렬 처리 구조의 성능을 확인하기 위해 수행하였다. 앞선 실험을 통해 Viterbi 계산과 external 분석 작업은 서로 다른 성격을 가진다는 점을 확인하였다. Viterbi 계산은 HMM 모델과 로그 데이터를 반복적으로 참조하며 분류 결과를 계산하는 작업으로, 동일한 주소 공간에서 데이터를 공유할 수 있는 thread 구조와 잘 맞는다. 반면 external 분석은 각 worker가 독립적인 분석 context를 기반으로 frequency, transition matrix, signature hashing, scratch buffer 등을 갱신하는 작업이므로 child process 기반 구조에 적합하다.

따라서 본 실험에서는 parent process, child process, worker thread를 모두 계산에 참여시키는 혼합 구조를 구성하였다. 특히 parent process도 단순히 child process와 thread를 생성한 뒤 대기하는 역할에 머무르지 않고, 하나의 worker로 직접 작업을 수행하도록 하였다. 이를 통해 parent process의 idle 시간을 줄이고, 가능한 많은 CPU core가 실제 계산에 참여하도록 설계하였다. 본 실험의 핵심 목적은 다음과 같다. 첫째, main process, child process, worker thread가 함께 동작하는 혼합 구조가 정상적으로 동작하는지 확인한다. 둘째, child process 수와 worker thread 수의 조합에 따라 실행 시간이 어떻게 달라지는지 비교한다. 셋째, 물리 core 수에 가까운 worker 구성이 실제로 가장 효율적인지 확인한다. 넷째, 각 worker가 균형 있게 작업을 수행하여 특정 worker에 병목이 발생하지 않는지 확인한다.

4.5.2 시스템 구성 및 동작 방식

본 실험에서 전체 worker 수는 다음과 같이 정의하였다.

W = parent process(main) + child process + worker thread

여기서 main은 항상 1개로 고정된다. Child process는 fork()를 통해 생성되는 독립 process이고, worker thread는 parent process 내부에서 pthread_create()를 통해 생성되는 thread이다. 따라서 main/child/thread = 1/3/1이라는 표기는 parent process가 1개, child process 3개, worker thread 1개가 계산에 참여한다는 의미이며, 전체 worker 수는 5개가 된다.

혼합 구조에서 child process는 external 분석 작업을 우선적으로 수행하도록 하였고, parent process와 worker thread는 Viterbi 계산을 우선적으로 수행하도록 하였다. 다만

특정 작업 queue가 먼저 비는 경우 worker가 idle 상태로 남지 않도록, 자기 우선 작업을 모두 처리한 worker는 반대 queue에 남아 있는 작업도 가져와 처리할 수 있도록 하였다. 이를 통해 Viterbi 작업과 external 작업의 수행 시간이 완전히 같지 않더라도 전체 worker가 가능한 한 오래 계산에 참여하도록 하였다.

작업 queue는 Viterbi 작업 queue와 external 작업 queue로 분리하였다. 각 queue는 다음에 처리할 로그 index를 가리키는 counter를 가지고 있으며, 여러 process와 thread가 동시에 접근하기 때문에 atomic fetch-and-add 방식으로 작업을 가져오도록 구현하였다. 이 방식은 하나의 로그가 중복 처리되거나 누락되는 문제를 방지한다. 또한 child process와 parent process는 서로 다른 주소 공간을 가지므로, 일반 heap 메모리만으로는 결과와 통계를 공유할 수 없다. 이를 해결하기 위해 작업 queue, 결과 배열, external score 배열, worker별 통계 구조체를 공유 메모리에 배치하였다. 이를 통해 child process가 처리한 결과도 parent process에서 정상적으로 집계할 수 있도록 하였다. 여러 thread를 먼저 생성한 뒤 fork를 수행하면 child process에 thread 관련 lock 상태가 불완전하게 복사될 수 있다. 이 경우 deadlock이나 예측하기 어려운 오류가 발생할 가능성이 있으므로 본 실험에서는 child process를 먼저 생성한 뒤, parent process 내부에서 worker thread를 생성하였다. 이후 parent process도 바로 worker loop에 진입하여 계산에 참여하도록 하였다.

4.5.3 실험 방법

실험은 총 10,000개의 로그를 대상으로 수행하였다. Viterbi 반복 계수는 R=100, external 반복 계수는 EXT_R=500으로 설정하였으며 입력 데이터로는 사전에 생성한 로그파일(./log_files_seed42/file_list.txt)을 사용하였다. 프로그램 실행 시 HMM 모델을 먼저 로딩하고, 이후 로그 목록을 읽어 전체 로그 데이터를 파싱하였다.

초기 로딩 결과는 다음과 같다.

항목	측정값
전체 로그 수	10,000개
HMM 모델 로딩 시간	0.001783초
로그 로딩 및 파싱 시간	0.553172초
Viterbi 반복 계수 R	100
External 반복 계수 EXT_R	500

전체 실행 시간이 수십 초 단위로 측정된 것과 비교하면 HMM 모델 로딩 시간과 로그 로딩 시간은 상대적으로 매우 작다. 따라서 본 실험에서 조합별 성능 차이는 대부분 병렬 작업 수행 구간에서 발생한다고 볼 수 있다.

측정한 조합은 다음과 같다.

조합	main	child process	worker thread	전체 worker 수(W)
1/1/1	1	1	1	3
1/2/1	1	2	1	4
1/1/2	1	1	2	4
1/1/3	1	1	3	5
1/2/2	1	2	2	5
1/3/1	1	3	1	5

각 조합에 대해 wall time, CPU 사용률, worker imbalance, Viterbi 처리 수, external 처리 수, 탐지 성공률, checksum을 확인하였다. 특히 실행 시간이 짧더라도 작업이 누락되면 올바른 결과로 볼 수 없기 때문에, Viterbi 작업과 external 작업이 각각 10,000개씩 모두 처리되었는지를 함께 확인하였다.

4.5.4 실험 결과 및 분석

실험 결과는 다음과 같다.

조합	W	실행 시간(s)	CPU 사용률(%)	Worker imbalance(s)	Viterbi 처리 수	External 처리 수	탐지 성공률
1/1/1	3	48.544878	297.70	0.011267	10,000	10,000	99.80%
1/2/1	4	29.504522	397.48	0.006281	10,000	10,000	99.80%
1/1/2	4	32.322566	396.31	0.006307	10,000	10,000	99.80%
1/1/3	5	23.135950	496.08	0.002964	10,000	10,000	99.80%
1/2/2	5	23.148262	495.98	0.003401	10,000	10,000	99.80%
1/3/1	5	23.084882	496.91	0.002772	10,000	10,000	99.80%

모든 조합에서 Viterbi 작업과 external 작업은 각각 10,000개씩 정상적으로 처리되었다. 또한 탐지 성공률은 모든 조합에서 99.80%로 동일하게 유지되었다. 따라서 조합별 차이는 정확도 차이가 아니라 병렬 처리 성능 차이에서 발생한 것으로 볼 수 있다.

가장 느린 조합은 1/1/1으로 이 조합은 parent process 1개, child process 1개, worker thread 1개로 구성되어 전체 worker 수가 3개이다. 실행 시간은 48.544878초였고, CPU 사용률은 297.70%로 측정되었다. 이는 약 3개의 CPU core를 활용한 결과로 볼 수 있다. 작업은 모두 정상적으로 처리되었지만, 전체 worker 수가 적어 병렬성이 충분히 확보되

지 못했기 때문에 실행 시간이 가장 길게 나타났다.

W=4 조합에서는 실행 시간이 크게 감소하였다. 1/2/1 조합은 29.504522초, 1/1/2 조합은 32.322566초로 측정되었다. 두 조합 모두 CPU 사용률은 약 396~397% 수준으로 나타났으며, 이는 약 4개의 CPU core를 활용했음을 의미한다. 1/2/1이 1/1/2보다 더 빠르게 나타난 이유는 external 분석 작업의 비중이 상대적으로 크기 때문에 child process를 2개로 늘린 구성이 더 유리했기 때문으로 해석할 수 있다.

W=5 조합에서는 실행 시간이 약 23초 수준으로 감소하였다. 1/1/3은 23.135950초, 1/2/2는 23.148262초, 1/3/1은 23.084882초로 측정되었다. 세 조합 모두 CPU 사용률이 약 496% 수준으로 나타났으며, 이는 물리 core 5개를 거의 모두 활용했음을 의미한다. 또한 worker imbalance도 0.003초 내외로 매우 작게 나타났다. 즉, 특정 worker가 오래 남아 병목이 되는 현상 없이 전체 worker가 거의 균등하게 작업을 수행하였다.

측정값만 기준으로 하면 가장 빠른 조합은 1/3/1이다. 1/3/1은 23.084882초로 전체 조합 중 가장 짧은 실행 시간을 보였고, CPU 사용률도 496.91%로 가장 높게 나타났다. Worker imbalance 역시 0.002772초로 가장 작았다. 따라서 본 실험에서 최종 최적 구성은 main/child/thread = 1/3/1, 즉 parent process 1개, child process 3개, worker thread 1개로 판단하였다. 다만 W=5에 해당하는 세 조합의 실행 시간 차이는 매우 작다. 1/3/1과 1/1/3의 차이는 약 0.051초이고, 1/3/1과 1/2/2의 차이는 약 0.063초이다. 전체 실행 시간이 약 23초인 점을 고려하면 0.2~0.3% 수준의 매우 작은 차이이다. 따라서 세 조합은 성능상 거의 유사하다고 볼 수 있으나 그럼에도 측정된 wall time, CPU 사용률, worker imbalance가 모두 가장 우수하기에 1/3/1을 최종 구성으로 선택하였다.

4.5.5 결과 분석

실험 결과에서 가장 뚜렷한 경향은 전체 worker 수 W가 증가할수록 실행 시간이 감소한다는 점이다. W=3인 1/1/1 조합은 48.544878초가 걸렸지만, W=4인 조합에서는 29~32초 수준으로 감소하였다. 이후 W=5인 조합에서는 약 23초 수준까지 감소하였다. 이는 worker 수가 증가하면서 더 많은 CPU core가 동시에 계산에 참여했기 때문이다. CPU 사용률도 이와 일치하는 경향을 보였다. W=3에서는 CPU 사용률이 297.70%였고, W=4에서는 약 396~397%, W=5에서는 약 496%로 증가하였다. 이는 worker 수가 증가할수록 실제 CPU core 활용도도 함께 증가했음을 의미한다. 특히 W=5 조합에서 CPU 사용률이 약 500%에 근접한 것은 실험 환경의 물리 core 5개를 거의 모두 활용했다는 의미로 해석할 수 있다. 또한 worker imbalance가 모든 조합에서 매우 작게 나타났다. 가장 큰 imbalance도 1/1/1의 0.011267초에 불과했고, 최적 구성인 1/3/1에서는 0.002772초로 측정되었다. 이는 각 worker가 거의 같은 시점에 작업을 마쳤다는 뜻이다. 즉, 특정

child process나 thread에 작업이 과도하게 몰려 전체 실행 시간이 지연되는 병목 현상이 거의 발생하지 않았다.

W=4 조합을 비교하면 child process를 2개 둔 1/2/1이 worker thread를 2개 둔 1/1/2 보다 더 빠르게 나타났다. 이는 external 분석 작업의 비중이 상대적으로 크거나, 해당 작업이 child process에서 더 효율적으로 처리되었기 때문으로 해석할 수 있다. 1/2/1은 29.504522초, 1/1/2는 32.322566초로 약 2.8초 차이를 보였다. 따라서 W가 4인 경우에는 child process를 늘리는 구성이 더 유리하였다. 그러나 W=5에서는 1/1/3, 1/2/2, 1/3/1의 차이가 매우 작았다. 이는 전체 worker 수가 물리 core 수와 일치하면서, 어떤 조합이든 CPU core를 거의 모두 활용할 수 있었기 때문으로 보인다. 이 구간에서는 child process와 worker thread의 세부 비율보다 전체 worker 수가 물리 core 수와 일치하는지가 더 중요한 요인으로 작용하였다. 다만 측정값 기준으로는 external 분석을 우선 수행하는 child process가 3개인 1/3/1이 가장 좋은 결과를 보였다.

최종적으로 1/3/1 조합이 가장 우수한 이유는 다음과 같이 해석할 수 있다. 전체 worker 수가 5개로 물리 core 수와 일치하여 CPU 자원을 거의 모두 활용하였으며, child process 3개가 external 분석 작업을 충분히 분산 처리하였다. 또한 parent process와 worker thread가 Viterbi 작업을 처리하면서, 남은 작업을 유연하게 보조 처리할 수 있어 전체 작업 균형이 유지되었고, worker imbalance가 매우 작아 특정 worker가 병목이 되지 않았으므로 이와 같은 우수한 성능을 보인 것을 유추해볼 수 있다.

4.5.6 최적 구성 도출

본 실험에서 최적 구성은 다음 기준으로 도출하였다. 첫째, child process와 worker thread가 모두 포함되어야 한다. 둘째, Viterbi 작업과 external 작업이 모두 정상적으로 처리되어야 한다. 셋째, 전체 실행 시간이 가장 짧아야 한다. 넷째, CPU 사용률이 물리 core 수에 가깝게 높아야 한다. 다섯째, worker imbalance가 작아야 한다. 이 기준을 적용하면 최종 최적 구성은 main/child/thread = 1/3/1이다. 이 구성은 parent process 1개, child process 3개, worker thread 1개로 구성된다. 전체 worker 수는 5개이며, 실험 환경의 물리 core 수와 일치한다. 또한 Viterbi 작업과 external 작업을 각각 10,000개씩 모두 처리하였고, 탐지 성공률도 99.80%로 유지되었다.

최종 선택한 구성의 결과는 다음과 같다.

항목	결과
최종 구성	main/child/thread = 1/3/1
Parent process	1개
Child process	3개
Worker thread	1개
전체 worker 수 W	5
실행 시간	23.084882초
CPU 사용률	496.91%
Worker imbalance	0.002772초
Viterbi 처리 수	10,000
External 처리 수	10,000
탐지 성공률	99.80%

이 구성은 모든 측정 조합 중 가장 짧은 실행 시간을 보였다. 또한 CPU 사용률이 496.91%로 측정되어 물리 core 5개를 거의 모두 활용하였고, worker imbalance도 0.002772초로 가장 작았다. 따라서 특정 worker에 작업이 치우치지 않고 전체 worker가 균등하게 활용된 것으로 볼 수 있다. 다만 W=5의 다른 조합들도 최적 구성과 거의 비슷한 성능을 보였다. 1/1/3은 23.135950초, 1/2/2는 23.148262초로 측정되어 1/3/1과의 차이가 매우 작았다. 따라서 본 실험의 핵심 결론은 1/3/1만이 압도적으로 우수하다는 것이 아니라, 전체 worker 수가 물리 core 수와 일치하는 W=5 구성이 전반적으로 가장 우수하다는 점이다. 그중 측정상 가장 좋은 결과를 보인 조합이 1/3/1이었기 때문에 최종 구성으로 선택하였다.

4.5.7 시행착오 및 트러블슈팅

첫 번째 시행착오는 실험 조합을 어떻게 제한할 것인지에 대한 문제였다. 처음에는 가능한 모든 child process와 worker thread 조합을 넓게 측정할 수 있었지만, 보고서에 사용하기에는 조합이 너무 많아 핵심 경향을 설명하기 어려웠다. 따라서 최종 실험에서는 1/1/1, 1/2/1, 1/1/2, 1/1/3, 1/2/2, 1/3/1의 6개 preset 조합을 선정하였다. 이 조합들은 worker 수가 3, 4, 5로 증가할 때의 성능 변화를 확인할 수 있고, child process와 worker thread의 비율 변화도 함께 비교할 수 있다는 장점이 있다.

두 번째 시행착오는 worker 수와 물리 core 수의 관계였다. worker 수를 많이 만들수록 성능이 계속 좋아질 것처럼 생각할 수 있지만, 실제로는 물리 core 수를 초과하면 context switching과 scheduling overhead가 증가할 수 있다. 본 실험 환경에서는 물리

core 수가 5개이므로, 최종 preset 조합도 W=5까지만 비교 대상으로 구성하였다. 실험 결과 W=5 조합에서 CPU 사용률이 약 496%까지 증가하였고, 실행 시간도 가장 짧게 나타났다. 이를 통해 물리 core 수에 맞춘 worker 구성이 가장 안정적임을 확인하였다.

세 번째 시행착오는 child process와 worker thread의 비율에 대한 문제였다. Viterbi 계산은 thread에 적합하고 external 분석은 child process에 적합하다고 판단하였지만, 실제 최적 비율은 작업량과 실행 환경에 따라 달라질 수 있다. 실험 결과 W=4에서는 1/2/1이 1/1/2보다 빨랐고, W=5에서는 1/3/1이 가장 빠르게 나타났다. 이를 통해 본 workload에서는 external 분석 작업의 비중이 작지 않으며, child process를 충분히 확보하는 구성이 좋은 성능을 낼 수 있음을 확인하였다.

네 번째 트러블슈팅은 공유 메모리 구성이다. Thread는 parent process와 같은 주소 공간을 사용하므로 결과 배열과 통계 구조체를 쉽게 공유할 수 있다. 그러나 child process는 fork 이후 독립된 주소 공간을 가지기 때문에 일반 heap에 저장된 결과를 parent process가 직접 확인할 수 없다. 이를 해결하기 위해 작업 queue와 결과 배열, worker 통계 구조체를 공유 메모리로 구성하였다. 이 구조를 통해 child process가 처리한 결과도 parent process에서 정상적으로 집계할 수 있었다.

다섯 번째 트러블슈팅은 fork와 pthread 생성 순서이다. 여러 thread가 생성된 뒤 fork를 수행하면 child process에는 fork를 호출한 thread만 복제되고, 다른 thread가 사용하던 lock이나 내부 상태가 불완전하게 남을 수 있다. 따라서 본 구현에서는 child process를 먼저 생성하고, 이후 parent process 내부에서 worker thread를 생성하였다. 이 순서를 통해 fork-after-thread 문제를 피하면서 안정적으로 혼합 구조를 실행할 수 있었다.

여섯 번째 트러블슈팅은 CPU pinning 문제이다. worker를 특정 CPU에 고정하면 성능이 안정될 수 있지만, 여러 조합을 연속으로 측정하는 preset mode에서는 이전 실행의 CPU affinity가 다음 실행에 영향을 줄 수 있다. 이 경우 특정 core만 사용하거나 CPU 활용률이 비정상적으로 낮아지는 문제가 발생할 수 있다. 따라서 본 실험에서는 내부 CPU pinning을 비활성화하고, 운영체제 scheduler가 전체 CPU를 자유롭게 활용하도록 하였다. 그 결과 W=5 조합에서 CPU 사용률이 약 500%에 근접하는 결과를 얻을 수 있었다.

일곱 번째 트러블슈팅은 결과 검증 방식이다. 병렬 처리에서는 실행 시간이 짧더라도 작업이 누락되거나 중복 처리되면 올바른 결과라고 볼 수 없다. 따라서 각 조합에 대해 Viterbi 처리 수, external 처리 수, 탐지 성공률, checksum을 함께 확인하였다. 모든 preset 조합에서 Viterbi와 external 작업이 각각 10,000개씩 처리되었고, 탐지 성공률도 99.80%로 동일하게 유지되었다. 이를 통해 성능 비교가 작업 누락이나 결과 오류로 인한 것이 아니라 실제 병렬 처리 성능 차이에 따른 것임을 확인하였다.

4.5.8 라 실험 결론

실험 (라)를 통해 parent process, child process, worker thread를 함께 사용하는 혼합 병렬 처리 구조의 성능을 확인하였다. 실험 결과 전체 worker 수가 증가할수록 실행 시간이 감소하였으며, CPU 사용률도 worker 수에 비례하여 증가하였다. 특히 W=5 조합에서는 CPU 사용률이 약 496%까지 증가하여 물리 core 5개를 거의 모두 활용하였다. 최종적으로 가장 좋은 성능을 보인 조합은 main/child/thread = 1/3/1이었다. 이 조합은 parent process 1개, child process 3개, worker thread 1개로 구성되며, 전체 worker 수는 5개이다. 실행 시간은 23.084882초로 가장 짧았고, CPU 사용률은 496.91%, worker imbalance는 0.002772초로 측정되었다. 또한 Viterbi 작업과 external 작업이 각각 10,000개씩 모두 처리되었고, 탐지 성공률도 99.80%로 유지되었다. 따라서 본 실험에서는 1/3/1을 최적 혼합 구성으로 선택하였다. 다만 W=5에 해당하는 1/1/3, 1/2/2, 1/3/1 세 조합의 성능 차이는 매우 작았다. 이는 전체 worker 수가 물리 core 수와 일치하면 child process와 worker thread의 세부 비율보다 전체 CPU core를 얼마나 충분히 활용하는지가 더 중요한 요인으로 작용했기 때문으로 해석된다.

결론적으로 본 실험의 핵심은 child process와 worker thread를 함께 사용하는 혼합 구조가 정상적으로 동작하며, parent process까지 worker로 참여시킬 경우 CPU idle time을 줄일 수 있다는 점이다. 또한 물리 core 수에 맞춰 worker 수를 구성하고, 남은 작업을 유연하게 분배하는 방식이 전체 성능 향상에 효과적임을 확인하였다.

4.6 실험 마. Synchronization 적용 유무 비교

실험 마에서는 실험 라에서 도출한 최적 혼합 시스템을 기준으로 synchronization 적용 유무에 따른 정확성과 성능 차이를 비교하였다. 실험 라의 preset mode 결과에서 가장 짧은 실행 시간을 보인 조합은 main 1개, child process 3개, worker thread 1개로 구성된 W=5 구조였다. 따라서 본 실험에서는 해당 구조인 W=5, C=3, T=1, steal=1 조건을 고정하고 synchronization을 적용한 경우와 적용하지 않은 경우를 비교하였다.

항목	값
전체 worker 수 W	5
child process 수 C	3
worker thread 수 T	1
parent main thread	worker로 참여
Tail stealing	ON
Viterbi 반복 계수 R	100
External 반복 계수 EXT_R	500
전체 로그 수	10,000
비교 대상	Synchronization 사용 / 미사용

synchronization이 필요한 핵심 공유 자원은 next_viterbi_index와 next_external_index이다. 두 변수는 각각 Viterbi 작업과 External 작업에서 다음으로 처리할 로그 index를 나타내는 공유 작업 queue 역할을 한다. parent main, child process, worker thread가 동시에 이 값을 읽고 증가시키므로, 해당 연산이 원자적으로 수행되지 않으면 race condition이 발생할 수 있다.

4.6.1 동기화 문제 발생 상황 유도

synchronization 적용 버전에서는 공유 index를 atomic fetch-and-add 방식으로 증가시켰다. 이를 통해 여러 worker가 동시에 작업 queue에 접근하더라도 각 worker가 서로 다른 로그 index를 가져가도록 구성하였다. 반면 synchronization 미적용 버전에서는 atomic 연산을 제거하고, 공유 index를 일반 read-modify-write 방식으로 증가시키도록 수정하였다.

```
static int fetch_next_index(volatile int *p) {
    int old = *p;
    sched_yield();
    *p = old + 1;
    return old;
}
```

위의 코드와 같이 sched_yield()를 read와 write 사이에 삽입하여 context switch 가능성을 높였고, 이를 통해 공유 index 갱신 과정에서 race condition이 드러나도록 구성하였다. 공유 index의 read-modify-write 과정이 원자적으로 보호되지 않으면 lost update가 발생하여 동일 로그가 중복 처리되거나 일부 로그가 누락될 수 있기에 이를 중점으로 구성하였다. critical section은 공유 작업 queue의 index를 읽고 증가시키는 구간으로, next_viterbi_index와 next_external_index는 모든 worker가 공통으로 접근하는 공유 변수이므로, 해당 변수에 대한 read-modify-write 연산은 임계 구역으로 분류된다.

Synchronization 미사용 버전에서는 이 임계 구역을 보호하지 않아 여러 worker가 같은 index를 동시에 읽는 race condition이 발생할 수 있으므로, Synchronization 적용 버전에서는 atomic fetch-and-add 연산을 사용하여 임계 구역을 하나의 원자적 연산으로 처리하였다.

공유 자원	Critical Section	문제 상황	발생 가능한 오류
next_viterbi_index	Viterbi 작업 index read-modify-write	여러 worker가 같은 index를 동시에 읽음	Viterbi 작업 중복 또는 누락
next_external_index	External 작업 index read-modify-write	여러 worker가 같은 index를 동시에 읽음	External 작업 중복 또는 누락

4.6.2 문제 해결 과정

Synchronization 미적용 버전에서는 공유 작업 index의 증가가 원자적으로 보장되지 않는다. 따라서 여러 process/thread가 동시에 같은 index를 가져가는 race condition이

발생할 수 있다. 이를 해결하기 위해 synchronization 적용 버전에서는 atomic fetch-and-add 연산을 사용하였다. Atomic 연산은 read, increment, write 과정을 하나의 원자적 연산으로 처리하므로, 여러 worker가 동시에 접근하더라도 동일한 index를 중복으로 가져가지 않는다.

Synchronization 적용 전후의 차이는 다음과 같다.

구분	Critical Section	공유 index 처리 방식	특징
Synchronization 미사용	공유 index read-modify-write	일반 증가 연산	race condition 발생 가능
Synchronization 사용	공유 index read-modify-write	atomic fetch-and-add	index 증가 연산의 원자성 보장

Synchronization 적용 버전에서는 각 로그가 Viterbi 작업과 External 작업에서 정확히 한 번씩 처리된다. 따라서 viterbi total count와 external total count는 전체 로그 수인 10,000과 일치해야 한다. 반면 synchronization 미적용 버전에서는 동일 로그가 중복 처리되거나 일부 로그가 누락될 수 있으므로, count 값과 checksum이 정상 버전과 달라질 수 있다.

4.6.3 동기화 적용 전후 성능 비교

동일한 W=5, C=3, T=1, steal=1 조건에서 synchronization 적용 버전과 미적용 버전을 실행한 결과는 다음과 같다.

항목	Synchronization 사용	Synchronization 미사용
전체 실행 시간(before cleanup)	23.084882 s	23.631414 s
전체 CPU 사용률 추정	496.91%	497.09%
Viterbi total count	10,000	10,004
External total count	10,000	10,007
악성코드 탐지 성공률	99.80%	99.84%
Viterbi checksum	24518236282129	24526128838767
External checksum	2979905468447192086	4435529050664965486
waitpid abnormal count	0	0
pthread failure count	0	0

실험 결과, synchronization을 사용한 경우 Viterbi 작업과 External 작업이 각각 정확히 10,000회 수행되었다. 이는 전체 로그 10,000개가 각 분석 단계에서 정확히 한 번씩 처리되었음을 의미한다. 또한 Viterbi checksum과 External checksum이 기준값으로 유지되어 작업 분배와 결과 집계가 정상적으로 이루어졌음을 보여준다.

반면 synchronization을 제거한 경우 Viterbi total count는 10,004, External total count는 10,007로 증가하였다. 전체 로그 수가 10,000개임에도 처리 횟수가 이를 초과했으므로, 동일한 로그 index가 여러 worker에게 중복 할당되었음을 알 수 있다. 이 결과는 공유 작업 index를 원자적으로 보호하지 않을 때 race condition이 실제로 발생함을 보여준다. 악성코드 탐지 성공률도 synchronization 사용 버전에서는 99.80%였으나, synchronization 미사용 버전에서는 99.84%로 증가하였다. 이 값은 실제 탐지 성능 향상이 아니라, 중복 처리된 로그가 success_count에 추가 반영되면서 발생한 통계적 왜곡이다. 따라서 synchronization 미사용 버전의 성공률은 정확한 성능 지표로 사용할 수 없다. Checksum에서도 차이가 나타났다. Synchronization 사용 버전의 Viterbi checksum은 245182362821290였으나, synchronization 미사용 버전에서는 24526128838767로 변경되었다. External checksum 역시 2979905468447192086에서 443552905066496548으로 달라졌다. 이는 처리된 작업의 중복 여부와 실행 흐름이 synchronization 적용 여부에 따라 달라졌음을 보여주는 검증 지표로서 사용된다. 실행 시간 측면에서도 synchronization 미사용 버전은 23.631414초로, synchronization 사용 버전의 23.084882초보다 더 오래 걸렸다. 동기화를 제거하면 atomic 연산의 비용은 사라지지만, 본 실험에서는 race condition으로 인해 중복 작업이 발생하였고, 그 결과 불필요한 Viterbi 및 External 연산이 추가로 수행되었다. 이로 인해 synchronization 미사용 버전의 실행 시간이 더 증가하였다.

실험 마를 통해 실험 라의 최적 hybrid 구조에서 synchronization이 필수적으로 필요함을 확인하였다. parent main, child process, pthread가 동시에 공유 작업 queue에 접근하는 구조에서는 작업 index 증가 연산이 critical section에 해당하며, 해당 구간은 반드시 원자적으로 수행되어야 한다. Atomic synchronization을 적용하면 각 로그가 정확히 한 번씩 처리되어 결과의 정확성과 재현성이 보장된다. 반면 synchronization을 제거하면 race condition으로 인해 작업 중복, count 값 증가, checksum 불일치, 실행 시간 증가가 발생한다. 따라서 본 프로젝트의 최종 hybrid 시스템에서 atomic 기반 synchronization이 정확성과 성능을 동시에 유지하기 위한 필수 요소임을 알 수 있다.

5. 결론

5.1 프로젝트 성과 요약

본 프로젝트는 운영체제에서 제공하는 child process와 thread 기반 병렬처리 기법을 HMM 기반 대량 로그 분석 시스템에 적용하고, 각 실행 구조의 성능과 정확성을 단계적으로 비교한 프로젝트이다. 전체 시스템은 10,000개의 로그를 입력으로 받아 Viterbi 연산과 External 연산을 수행하도록 구성하였다. 두 연산은 모두 CPU-bound 특성을 가지며, 로그 단위로 독립적으로 처리할 수 있어 병렬처리 실험에 적합한 구조를 가진다.

실험 가에서는 parent process 단독 순차 실행과 child process 병렬 실행을 비교하였다. 순차 실행에서는 하나의 실행 흐름이 전체 로그를 처리하므로 CPU core를 충분히 활용하지 못하였다. 반면 child process를 도입한 병렬 구조에서는 worker 수 증가에 따라 실행 시간이 크게 감소하였다. 이를 통해 로그 단위 분석 작업이 병렬화에 적합하며, child process 기반 병렬처리만으로도 순차 실행 대비 유의미한 성능 향상을 얻을 수 있음을 확인하였다.

실험 나에서는 단일 child process와 단일 thread를 각각 최적화된 구조로 구현하고, 이를 교차 비교하였다. Viterbi 연산은 HMM 모델과 로그 배열을 read-only로 반복 참조하므로 공유 주소 공간을 활용할 수 있는 thread에 적합하였다. 반면 External 연산은 독립적인 분석 context를 유지하면서 내부 상태를 갱신하는 특성이 있어 process의 독립 주소 공간을 활용하는 구조에 적합하였다. 이 실험을 통해 단순히 process와 thread 중 어느 하나가 항상 우수한 것이 아니라, 작업의 메모리 접근 방식과 상태 관리 방식에 따라 적합한 실행 단위가 달라짐을 확인하였다.

실험 다에서는 child process와 thread 각각을 다중 worker 구조로 확장하여 성능 변화를 분석하였다. worker 수가 증가할수록 전체 실행 시간은 감소하고 CPU 사용률은 증가하였다. 다만 물리 core 수에 가까워질수록 메모리 대역폭 경쟁, cache 영향, scheduling overhead 등으로 인해 이상적인 선형 speedup에는 완전히 도달하지 못하였다. 이를 통해 병렬처리 성능은 worker 수만으로 결정되는 것이 아니라, 하드웨어 자원과 작업 분배 방식의 영향을 함께 받는다는 점을 확인하였다.

실험 라에서는 앞선 실험 결과를 바탕으로 child process와 thread를 혼합한 hybrid 구조를 설계하였다. Parent main thread도 worker로 참여하도록 하여 idle time을 줄였고, child process는 External 작업을 우선 처리하며, worker thread는 Viterbi 작업을 우선 처리하도록 역할을 분리하였다. 또한 tail stealing을 적용하여 한쪽 작업 queue가 먼저 비는 경우에도 worker가 반대 queue의 작업을 가져와 계속 계산을 수행하도록 하였다. 실험 결과 main/child/thread = 1/3/1 구조가 가장 짧은 실행 시간을 보였으며, Viterbi 작업

과 External 작업 모두 10,000개씩 정상적으로 처리되었다. 이를 통해 process와 thread를 작업 특성에 맞게 혼합하면 단일 방식보다 더 효율적인 병렬 시스템을 구성할 수 있음을 확인하였다.

실험 마에서는 실험 라의 최적 hybrid 구조에서 synchronization 적용 유무를 비교하였다. Hybrid 구조에서는 parent main, child process, worker thread가 동시에 공유 작업 queue에 접근하므로 next_viterbi_index와 next_external_index를 갱신하는 구간이 critical section에 해당한다. Synchronization을 제거한 경우 Viterbi total count와 External total count가 10,000을 초과하여 동일 로그가 중복 처리되는 문제가 발생하였다. 반면 atomic fetch-and-add를 적용한 synchronization 사용 버전에서는 각 로그가 정확히 한 번씩 처리되었다. 이를 통해 병렬 시스템에서 synchronization은 단순한 보조 기능이 아니라 결과의 정확성과 재현성을 보장하기 위한 필수 요소임을 확인하였다.

최종적으로 본 프로젝트에서는 순차 실행에서 출발하여 child process 병렬화, process/thread 구조 비교, 다중 worker 확장, hybrid 구조 최적화, synchronization 문제 해결까지 단계적으로 수행하였다. 그 결과 병렬처리 성능 향상은 실행 단위의 개수뿐만 아니라, 작업 특성에 맞는 process/thread 역할 분담, 작업 분배 방식, 공유 자원 보호 방식에 의해 결정됨을 확인하였다. 또한 운영체제에서 학습한 process, thread, IPC, scheduling, critical section, synchronization 개념이 실제 프로그램의 성능과 정확성에 직접적인 영향을 미친다는 점을 실험적으로 확인하였다.

5.2 한계 및 개선 방향

본 프로젝트는 운영체제 병렬처리 구조의 성능 분석을 주된 목적으로 하였기 때문에, HMM 기반 악성코드 탐지 모델 자체는 실제 보안 시스템 수준으로 정교하게 설계하지 않았다. 로그 데이터와 HMM 모델은 병렬처리 실험을 위한 workload로 구성되었으므로, 본 보고서의 탐지 성공률은 상용 악성코드 탐지기의 정확도를 의미하기보다는 병렬 실행 구조에서 결과가 일관되게 유지되는지를 검증하기 위한 지표로 해석하는 것이 적절하다.

또한 실험은 제한된 하드웨어 환경에서 수행되었다. 본 프로젝트에서는 물리 core 수에 가까운 worker 수에서 가장 안정적인 성능을 보였지만, 더 많은 core를 가진 서버 환경이나 NUMA 구조에서는 최적 worker 수와 process/thread 조합이 달라질 수 있다. 따라서 다양한 하드웨어 환경에서 추가 실험을 수행하면 병렬 구조의 일반성을 더 명확하게 검증할 수 있다.

동기화 측면에서는 공유 작업 index에 대해 atomic fetch-and-add를 적용하여 race condition을 해결하였다. 이 방식은 단순 counter 기반 작업 queue에는 적합하지만, 향후

작업 queue가 복잡한 자료구조로 확장되거나 공유 model 갱신이 더 자주 발생하는 경우 mutex, semaphore, condition variable, lock-free queue 등 다양한 synchronization 기법의 비교가 필요하다.

마지막으로, 현재 시스템은 로그 단위 작업의 독립성을 전제로 설계되었다. 실제 로그 분석 시스템에서는 로그 간 시간적 관계나 프로세스 간 상호작용을 함께 고려해야 할 수 있다. 이러한 경우 작업 간 의존성이 생기므로, 단순한 로그 단위 병렬화보다 더 정교한 scheduling 및 synchronization 설계가 필요하다.

5.3 향후 확장 계획

향후에는 실제 system call trace 데이터를 적용하여 시스템의 현실성을 높일 수 있다. 실제 악성코드와 정상 프로그램에서 수집한 로그를 사용하면 HMM 모델의 탐지 성능과 병렬처리 구조의 실용성을 함께 검증할 수 있다. 또한 worker 배정 전략을 개선할 수 있다. 본 프로젝트에서는 role-based queue와 tail stealing을 사용하여 Viterbi 작업과 External 작업의 부하를 분산하였다. 이후에는 로그 길이, 예상 연산량, worker별 처리 속도 등을 반영한 adaptive scheduling 방식을 적용하여 더 정교한 load balancing을 구현할 수 있다.

동기화 방식 역시 확장 가능하다. 현재는 공유 작업 index에 대한 atomic 연산을 중심으로 실험하였지만, 실제 시스템에서는 shared model, result buffer, dynamic queue 등 다양한 공유 자원이 존재할 수 있다. 향후에는 atomic, mutex, semaphore, read-write lock, lock-free 자료구조를 동일 workload에서 비교하여 각 방식의 적용 조건과 성능 차이를 분석할 수 있다.

마지막으로, 본 프로젝트의 구조는 악성코드 탐지뿐만 아니라 대량 로그 분석, 서버 모니터링, 보안 관제 시스템, 이벤트 스트림 처리 시스템에도 확장할 수 있다. 로그 단위 독립 연산과 일부 공유 상태 갱신이 함께 존재하는 시스템이라면, 본 프로젝트에서 도출한 process/thread 역할 분리, hybrid worker 구성, synchronization 설계 방식이 유사하게 적용될 수 있다.

6. 팀 운영

6.1 역할 분담

본 프로젝트는 실험 단계(가~마)를 기준으로 팀원별 담당을 분배하였다. 각 팀원은 담

당 실험의 코드 구현·측정·분석과 해당 보고서 절의 작성을 책임지며, 공통 작업(실험 환경 구성, 발표자료)은 전원이 분담하였다.

팀원	담당 실험	세부 역할
박진서	(가) 순차 vs 병렬 (나) 단일 process vs thread	exp_a 순차/child fulltask 구현, round-robin 분배 및 baseline 측정. 나번 B1~B4 work-conserving 역할분리 구현, process-친화/thread-친화 코드 설계 및 교차 비교, hmm_models 생성
안민석	(다) 다중 process/thread 확장	단일 구조를 다중 worker로 확장하는 proc_multi/thread_multi 구현 및 수정, worker 수 확장성(speedup·efficiency) 측정, 최적 worker 수 도출
남윤겸	(라) hybrid 시스템 통합	다번 결과를 바탕으로 process와 thread를 결합한 hybrid 시스템 설계, worker 3~5 조합 실험, 작업 성격별 실행 단위 배정, stealing 동작 설계
이건아	(마) synchronization	라번 최적화 시스템에 대한 동기화 가능 구역 식별, synchronization 문제 유도 및 해결, 동기화 적용 전후 성능 비교, IPC 설계

6.2 팀원별 기여도 자체 평가

팀원별 담당 실험의 난이도, 코드 구현 분량, 보고서 작성량, 수행일지에 기록된 활동을 종합하여 기여도를 차등 평가하였다. 합계는 100%이다.

팀원	담당	기여도	평가 근거
박진서	(가)(나)	30%	가·나번 두 실험을 단독 담당하여 전체 baseline과 work-conserving 구조의 기반을 확립. 코드 전면 수정 및 실험 재설계를 주도
남윤겸	(라)	26%	hybrid 시스템 통합 및 다양한 worker 조합 실험 수행, 전체 실험 환경·설계를 주도적으로 기획
안민석	(다)	24%	다중 확장 코드 구현 및 확장성 분석, 성능 병목의 원인 규명과 최적점 도출 담당
이건아	(마)	20%	동기화 문제 유도·해결 및 IPC 설계, 동기화 전후 성능 비교 담당

합계: 박진서 30% + 남윤겸 26% + 안민석 24% + 이건아 20% = 100%

6.3 수행 계획서

단계	기간	주요 내용	산출물
주제 선정·구상	~04/19	주제 후보 발표 및 HMM 기반 악성코드 로그분석 확정, 역할 분담, 아키텍처 구체화	수행계획서 발표
Baseline 구현	~05/03	실험 환경 구성, log_files·hmm_models 생성, 가·나번(순차/병렬, 단일 process/thread) 구현	중간보고서·발표
다중 확장·최적화	~05/28	다번 다중 worker 확장, CPU 사용률·load imbalance 문제 분석, 성능 병목 규명	proc_multi/thread_multi
Hybrid·동기화	~05/31	라번 hybrid 통합, 마번 synchronization 구역 설계 및 문제 해결	최종 코드
보고서·발표	~06/07	전체 결과 통합, 동일 세션 재측정, 설계보고서 및 발표자료 작성	최종 설계보고서

6.4 프로젝트 수행 일지

정기 모임에서 각 팀원이 진행 상황을 발표하고 다음 단계를 협의하였다. 주요 일자별 활동은 다음과 같다.

일자	주요 활동 요약
04/09	주제 브레인스토밍 — 각자 주제 후보 발표(콘서트 예매, 로그 병렬처리, 이미지 컨볼루션, 악성코드 스캐너 등)
04/17	아키텍처 구체화 — worker thread 동작, 시그니처 병렬 매칭, 생산-소비자 모델, IPC·동기화 방안 설계
04/25	평가 환경 모델링 — RAM 디스크 활용, parent/child 통신 정의, process/thread 동작 차이 검토
04/29	가번 설계 — parent 순차 실험 설계·구현, 병렬 실행 코드 작성 및 실행, 문제 파악
05/02	중간발표 준비 — 실험 환경 구성, log_files·hmm_models 설계, 시드별 실험 방안 제안
05/03	중간발표 — 순차/병렬 코드 실행 및 발표자료 제작
05/25	CPU 사용률 최적화 — stealing 0/1 상황 설계, load imbalance 해결 실험 조건 제시, 코드 전면 수정
05/28	원인 분석 — Viterbi 연산에서 process 이점 부재 발견 및 원인 분석, external 연산 도입으로 문제 해결
05/31	역할 확정 및 확장 — 라(남윤겸)/마(이건아) 분담, 다번 다중 확장 코드 설계·수정, process/thread 친화 코드 재설계
~06/07	최종 통합 — 동일 세션 재측정, 결과 분석 정리, 보고서·발표자료 작성, 기여도 평가

6. 사용한 AI agent

본 프로젝트에서는 실험 코드 작성 및 수정 과정에서 ChatGPT(Codex), Claude code를 AI agent로 활용하였다. 이러한 AI agent는 프로세스와 스레드 기반 병렬 처리 구조를 구현하는 과정에서 코드 구조 설계, 실행 옵션 구성, 성능 측정 항목 추가, 오류 원인 분석을 보조하는 용도로 사용하였다. 단, AI가 생성하거나 제안한 코드는 그대로 사용하지 않고, 직접 코드 동작을 검토하고 실험 환경에 맞게 수정하여 최종 실험에 사용하였다.

입력 Prompt 예시:

- 내가 제시해준 코드에서 main/child process/thread 조합을 1/1/1, 1/2/1, 1/1/2, 1/1/3, 1/2/2, 1/3/1 같이 자동으로 수행할 수 있도록 하는 스크립트를 작성해줘. (Claude)
- 지금 내 로컬 디렉터리에 있는 C코드를 수정해서 speedup과 efficiency 값을 자동으로 계산해서 출력하는 동작을 추가시켜줘. (Codex)